

version0.2.0

FactVATE

セットアップガイド

nogikun

1. はじめに

この文書は、プロトタイプのセットアップ手順について記載したものです。他の開発者や関係者が同様の環境を再現できるようにすることを目的としています。

2. 前提条件

以下のソフトウェアおよびツールが必要です：

OS (アプリ)	• Android 14+ • iOS 15+
OS (システム)	• Windows 10 Pro/11 Pro • macOS 15+ (Sequoia) • Ubuntu 22.04 LTS
開発環境	• Docker Desktop (インストールガイドはこちら) • Node.js 18+ (インストールガイドはこちら) • Python 3.10 • Android Studio • Xcode
その他 依存ライブラリ	• Git • Docker, Docker-Compose • Rye (インストールガイドはこちら) • ngrok (セットアップ方法はこちら)

※ また iOS 端末への書き出しは Xcode に登録されている開発者プロファイルを iPhone で許可している必要がある。

3. セットアップ手順

以下の手順に従って環境を構築してください：

3.1. 「.env」ファイルの定義

1. `/dev/server/api/` 内に `.env` というファイルを作成する。
2. Gemini の API キーを取得する。
3. 以下の内容を記入。

<code>.env</code> (<code>/dev/server/api/.env</code>)
<code>DATABASE_URL="postgresql+psycopg2://postgres:postgres@postgres:5432/gamification"</code>
<code>GEMINI_API_KEY="ここに API_KEY を貼り付ける"</code>

※ ここに API_KEY を貼り付ける を変更する

3.2. サーバー / 企業向け管理コンソールのセットアップ

1. 「server」ディレクトリへ移動
以下のコマンドを実行し、該当ディレクトリへ遷移してください。

<code>PowerShell</code> <code>PS> cd dev/server</code> <code>```</code>	<code>Bash</code> <code>\$ cd dev/server</code> <code>```</code>
--	--

2. 必要パッケージをインストールする (frontend)

下記のコマンドを順に実行することで必要パッケージをインストールすることができる。

※インストール後は、`/dev/server/frontend`内に`node\_modules`というフォルダが生成されているはず。

<pre>PowerShell PS> cd frontend PS> npm install PS> cd .. '''</pre>	<pre>Bash \$ cd frontend \$ npm install \$> cd .. '''</pre>
--	--

3. 必要パッケージをインストールする (backend)

※インストール後は、`/dev/server/backend/api`内に`.venv`というフォルダが生成されているはず。

<pre>PowerShell PS> cd backend/api PS> rye sync '''</pre>	<pre>Bash \$ cd backend/api \$ rye sync '''</pre>
---	---

3.3. アプリケーションのセットアップ

本セクションでは、各 OS の書き出し手順について記載する

3.3.1. 求職者向けアプリのセットアップ(Android)

1. 「app」ディレクトリへ移動

以下のコマンドを実行し、該当ディレクトリへ遷移してください。

<pre>PowerShell PS> cd dev/app '''</pre>	<pre>Bash \$ cd dev/app '''</pre>
---	-----------------------------------

2. 必要パッケージをインストールする

<pre>PowerShell PS> npm install '''</pre>	<pre>Bash \$ npm install '''</pre>
--	------------------------------------

3. アプリケーションをビルドする

以下のコマンドを実行し、Android Studio 用のプロジェクトを書き出す。

<pre>PowerShell PS> npm run android '''</pre>	<pre>Bash \$ npm run android '''</pre>
--	--

4. Android Studio で`/dev/app/android`を開く

※ 以下のコマンドを実行しても開くことができる

<pre> PowerShell PS> ionic cap open android </pre>	<pre> Bash \$ ionic cap open android </pre>
---	---

5. テスト機(Android 端末)を実行 PC とデバックモードで接続し、書き出しを行う。
接続している端末が表示されているか確認し、
図中の「ビルドして実行」を押下し、アプリケーションを書き出す

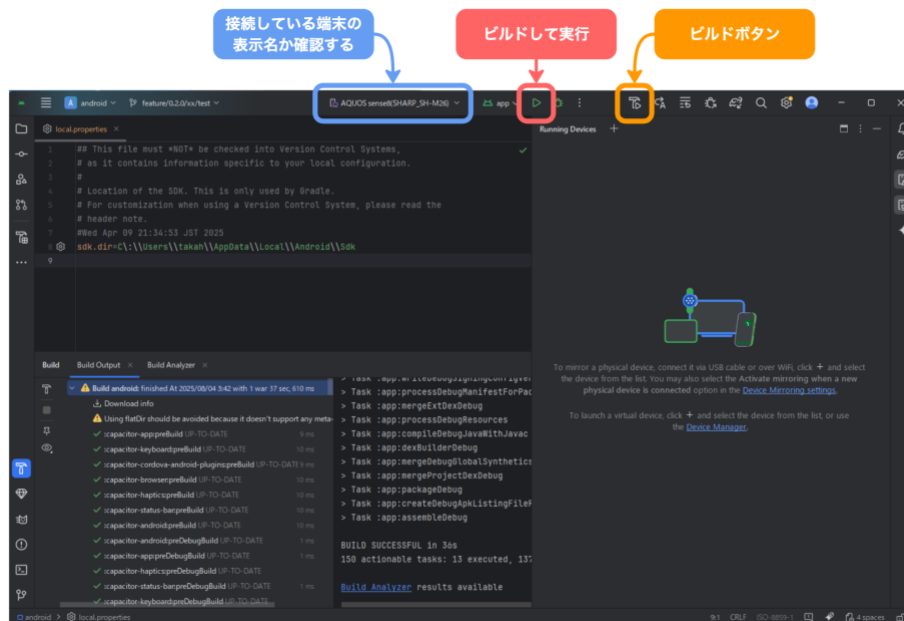


図 1 : Android Studio 書き出し画面

3.3.1. 求職者向けアプリのセットアップ(iOS)

1. 「app」ディレクトリへ移動

以下のコマンドを実行し、該当ディレクトリへ遷移してください。

<pre> PowerShell PS> cd dev/app </pre>	<pre> Bash \$ cd dev/app </pre>
---	---------------------------------

2. 必要パッケージをインストールする

<pre> PowerShell PS> npm install </pre>	<pre> Bash \$ npm install </pre>
--	----------------------------------

3. アプリケーションをビルドする

以下のコマンドを実行し、Android Studio 用のプロジェクトを書き出す。

```
```PowerShell
PS> npm run ios
```
```

```
```Bash
$ npm run ios
```
```

4. Xcode で `/dev/app/ios` を開く

※ 以下のコマンドを実行しても開くことができる

```
```PowerShell
PS> ionic cap open ios
```
```

```
```Bash
$ ionic cap open ios
```
```

5. テスト機(iOS 端末)を実行 PC と接続し、書き出しを行う。

※前提：iOS 端末で Xcode に登録されている開発者プロファイルを許可している事

接続している端末が表示されているか確認し、

図中の「ビルドして実行」を押下し、アプリケーションを書き出す

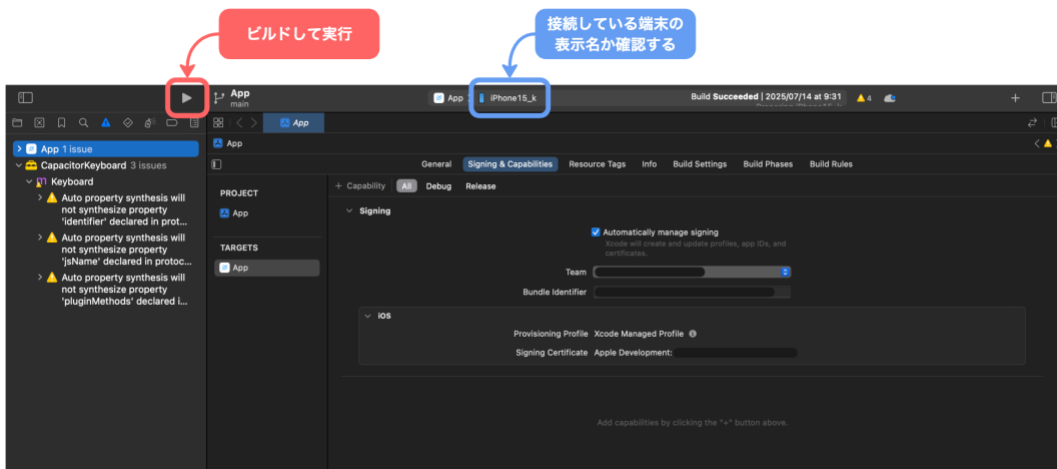


図 2 : Xcode 書き出し画面

4. 実行手順

本システムは「工場向け管理コンソール」と「求職者向けアプリ」の二種存在する。
そのため、以下の

4.1. サーバー / 工場向け管理コンソールの起動

1. server ディレクトリへ移動

以下のコマンドを実行し、該当ディレクトリへ遷移してください。

```
``PowerShell
PS> cd dev/server
``
```

```
``Bash
$ cd dev/server
``
```

2. Docker の実行

以下のコマンドを実行する事でシステムを起動でき、各コンテナの実行ログが閲覧できます。

```
``PowerShell
PS> docker-compose build
PS> docker-compose up
``
```

```
``Bash
$ docker-compose build
$ docker-compose up
``
```

※ 本資料と異なるインストール方法で環境を用意する場合は、
`docker-compose` ではなく `docker compose` にて動作する場合があります。

3. 各サービスにアクセスする

以下の表を参考にアクセスしてください。

企業がアクセスするのは、管理コンソールのみ。

| サービス名 | Port | URL |
|------------------|------|---|
| 管理コンソール | 5173 | http://localhost:5173 |
| API (Swagger) | 3000 | http://localhost:3000/docs |
| データベース (adminer) | 5050 | http://localhost:5050 |

※ データベースの認証は下記の情報を利用ください

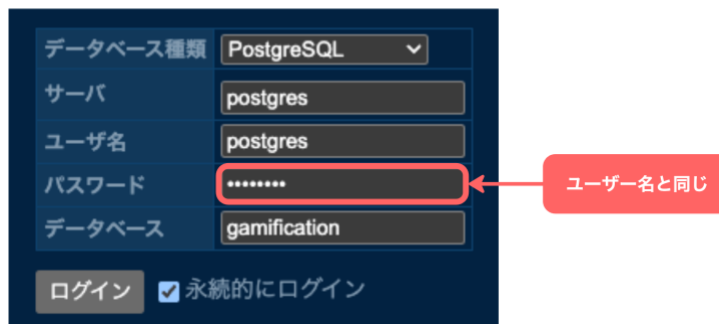


図 3 : adminer 認証画面

4.2.アプリケーションの実行

1. インストールされたアプリを開く

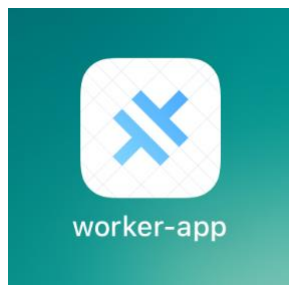


図4：アプリアイコン（ホーム画面）

2. アプリ内「設定」より「接続先サーバー」を任意のものへ変更し保存する

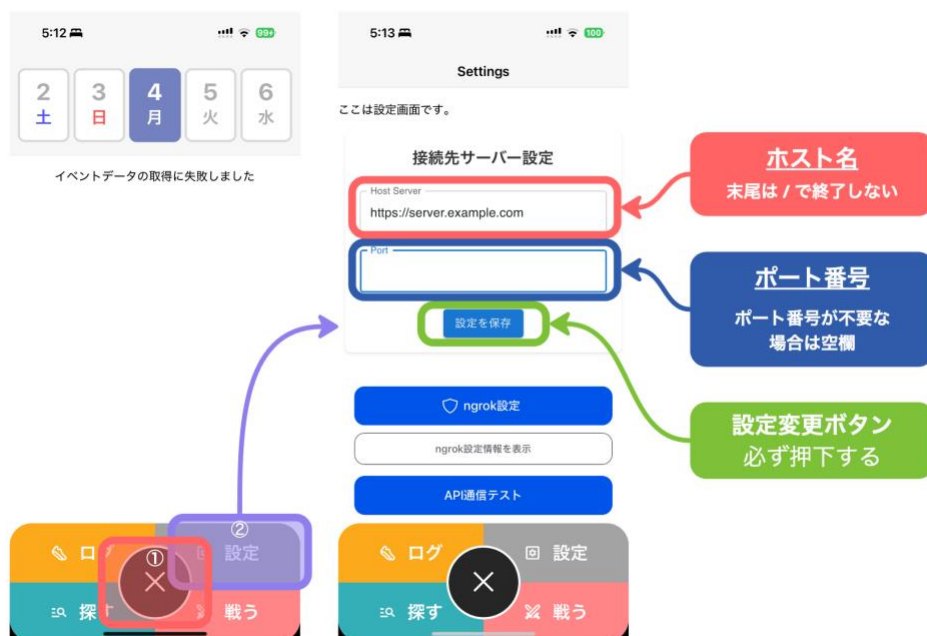


図5：ホスト・ポート情報変更

3. アプリケーションを利用する。

5. トラブルシューティング

よくあるエラーとその対処法を記載してください。

5.1. 求職者向けアプリケーションにおいて、Linux 環境で npm install を実行する際に、rollup 関連のパッケージが正常にインストールできない問題が発生する場合がある

```
npm install
npm ERR! code EBADPLATFORM
npm ERR! notsup Unsupported platform for @rollup/rollup-win32-x64-msvc@4.44.2: wanted {"os":"win32","cpu":"x64"} (current: {"os":"darwin","cpu":"arm64"})
npm ERR! notsup Valid os: win32
npm ERR! notsup Actual os: darwin
npm ERR! notsup Valid cpu: x64
npm ERR! notsup Actual cpu: arm64
npm ERR! A complete log of this run can be found in: /Users/takahashikazuaki/.npm/_logs/2025-08-03T18_58_56_783Z-debug-0.log
```

図 6 : 5.1. npm install エラー画面

| | |
|------|---|
| 対処方法 | この問題を解決するために、以下の手順を実行してください
1. `dev/app/package.json`と`dev/server/frontend/package.json`ファイルを開く
2. 各ファイルで、「win32」と記載のあるパッケージを特定し、それをファイルから削除する。 |
| 原因 | この問題は、Windows 環境で npm install を実行した際に、Windows 固有の依存ライブラリが自動的に package.json に記載されることが原因です。
これらの Windows 専用パッケージが Linux 環境では対応していないため、インストール時にエラーが発生します。 |

5.2. Docker の実行ができない

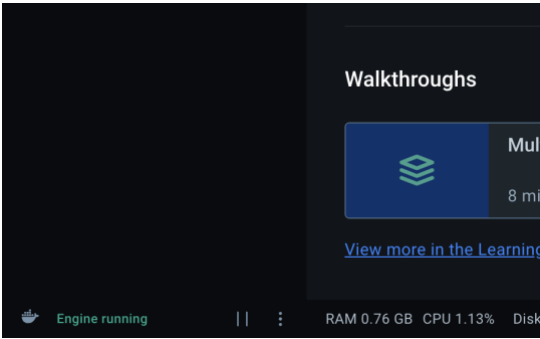
| | |
|------|---|
| 対処方法 | Docker Desktop を起動し、左下のステータスが「Engine running」である事を確認してください。
 |
| 原因 | Docker の demon が実行されていない。（待機状態でない） |

図 7 : Docker Desktop ウィンドウ 左下

5.3. iOS の書き出し中、「信頼されていないデベロッパ」と警告を受けビルドできない

| | |
|------|-----------------------|
| 原因 | iPhone で開発者を信頼できていない。 |
| 対処方法 | |

以下の手順で、開発者を信頼する。



図 8 : iOS にてデベロッパを信頼する手順

5.4. Android アプリケーション、サーバー間で通信に失敗する

| | |
|------|--|
| 対処方法 | <p>Ngrok 等のポートフォワーディングサービスを利用し、クライアント端末から見てセキュアな通信にする。</p> <p>詳細は、補足情報「6.4. ngrok によるポートフォワーディングの方法」を参照してください。</p> <pre> ngrok 🔓 Using ngrok for OSS? Request a community license: https://ngrok.com/r/oss Session Status online Account username (Plan: Free) Version 3.25.1 Region Japan (jp) Web Interface https://2b710101217810 Forwarding https://xxx.ngrok-free.app > http://localhost:3000 Connections ttl opn rt1 rt5 p50 p90 0 0 0.00 0.00 0.00 0.00 </pre> <p>公開URL</p> |
| 原因 | Android の場合セキュリティの関係で `http://` の通信がブロックされる為 |

図 9 : ターミナル (フォワーディング済み)

6. 補足情報

その他の注意点や参考資料を記載する。

6.1. Docker-Desktop のインストール方法

1. Docker の公式ページ (<https://www.docker.com/get-started/>) よりインストーラーをダウンロードしてください。

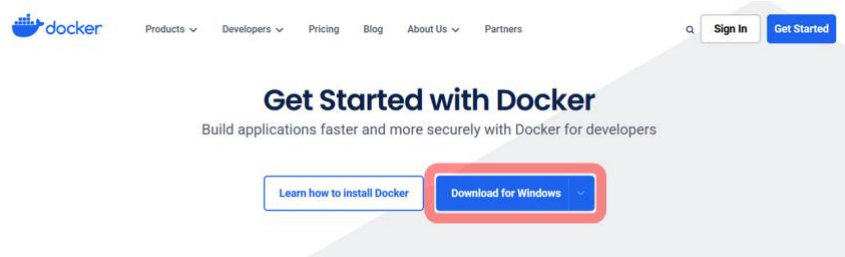


図 10 : Docker 公式ページ (<https://www.docker.com/get-started/>)

2. インストーラーのコンフィグレーションは下記のようにしてください。
※ Windows のサービス「Hyper-V」を有効にする必要があります。

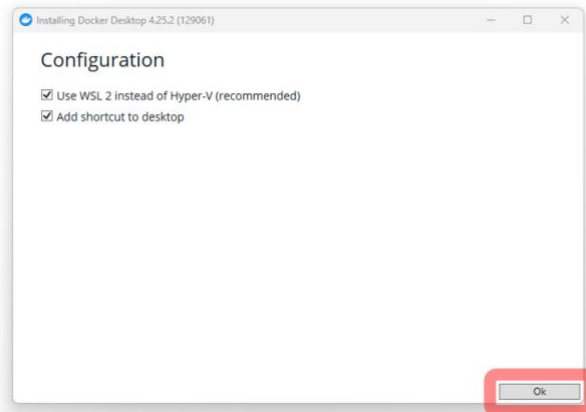


図 11 : Docker-Desktop インストーラー画面

3. インストール後、再起動をし、ターミナルから以下のコマンドを実行してください。
ソフトウェアのバージョンが出力されるとインストールされたことを確認することができます。

```
PowerShell
PS> docker -v
PS> docker-compose -v
```
```

```
bash
$ docker -v
$ docker-compose -v
```
```

6.2. Rye のインストール方法

Python のパッケージ管理ツールとして Rye を利用しています。
システムの実行に必要なのでインストールしてください。

【note】

Rye は、Python のパッケージ管理だけでなく、Python のバージョン管理や、ライブラリの高速インストール、仮想環境の管理なども行えるツールです。

※ 利用方法は 記事「[Python ライブラリ管理ツール決定版！Rye を導入してみた](#)」を参照してください。

macOS / Ubuntu の場合：

1. ターミナルで以下のコマンドを入力することで、環境に Rye をインストールすることができます。

```
```bash
$ curl -sSf https://rye.astral.sh/get | bash
```
```

2. ターミナルを再起動し、`rye --version` を実行することで環境がインストールされたことを確認できます。

Windows の場合：

1. [Rye のインストール](#) から下図のようなページにアクセスする事ができます。
上記の画面内、`rye-x86\_64-windows.exe` をクリックしてダウンロードしてください。ダウンロードしたファイルを実行し、インストールしてください。

参考資料：[Rye とは？【概要・環境構築・使い方】](#)

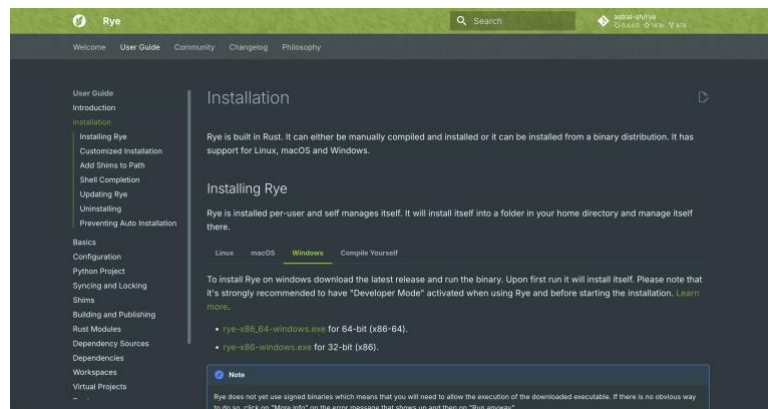


図 12：Rye 公式インストールガイド(<https://rye.astral.sh/guide/installation/>)

2. インストール後、Windows の Path に `%USERPROFILE%\rye\shims` を追加してください。
3. ターミナルを再起動し、`rye --version` を実行することで環境がインストールされたことを確認できます。

6.3. Node.js のインストール方法

下記 URL から Node.js をインストールしてください。

Node.js : <https://nodejs.org/ja/download/>

1. [Node.js のインストール](https://nodejs.org/ja/download/) から下図のようなページにアクセスする事ができます。
LTS 版を選択し、それぞれの OS に合ったインストーラーをダウンロードしてください。



図 13 : Node.js ダウンロードページ (<https://nodejs.org/ja/download/>)

2. インストール後、ターミナルを開き、`node -v` を実行することでインストールされたことを確認することができます。

6.4. ngrok によるポートフォワーディングの方法

1. ngrok 公式ページ (<https://ngrok.com/>) よりアカウント登録 / ログインを行う。

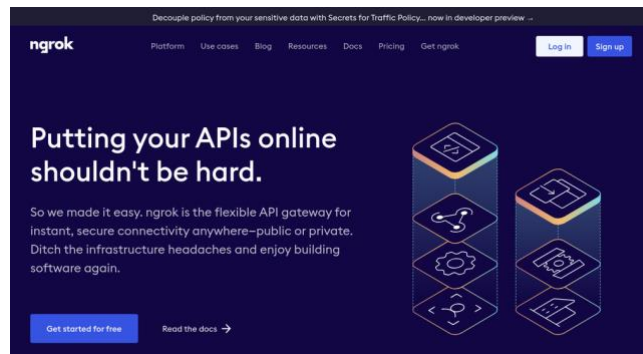


図 14 : ngrok 公式ページ (<https://ngrok.com/>)

2. ダッシュボードの「[Setup & Installation](#)」ページよりクライアントツールのインストールを行う

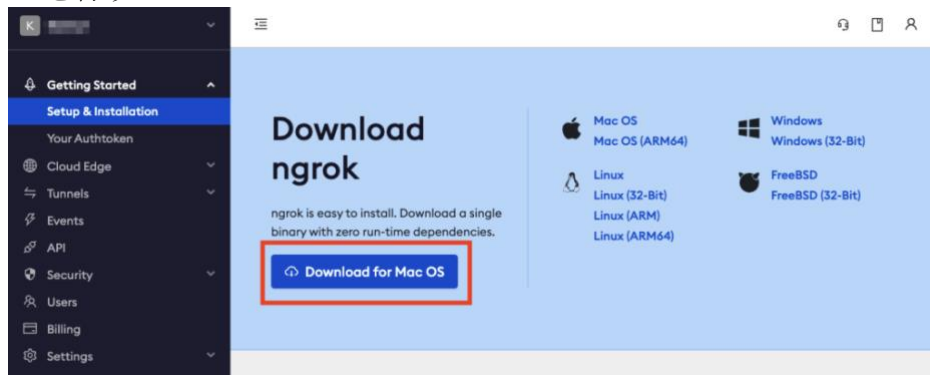


図 15 : <https://dashboard.ngrok.com/get-started/setup/macos>

3. ngrok を初めて使用する際には、認証トークンを設定する必要があります。これは、
ngrok ダッシュボードの「[Your Authtoken](#)」ページで確認できます。また、以下のコマンドで認証トークンを設定します。

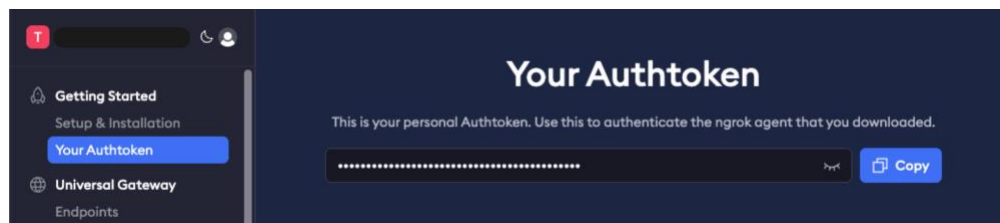


図 16 : <https://dashboard.ngrok.com/get-started/your-authtoken>

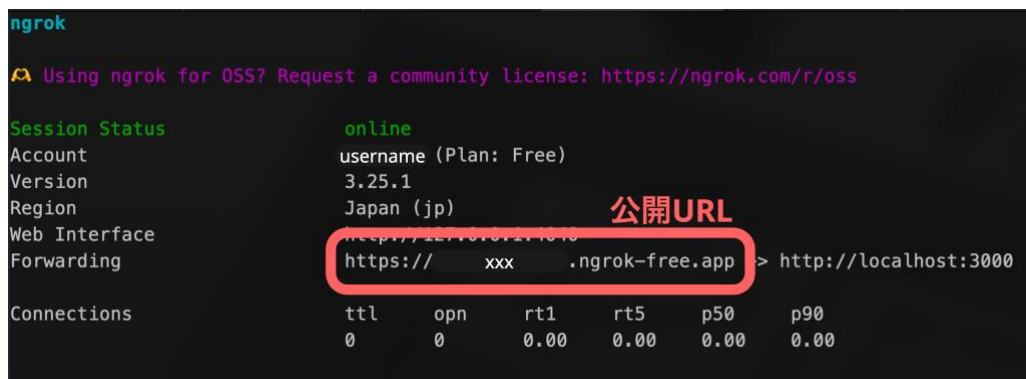
```
""PowerShell
PS> ngrok config add-authtoken <YOUR_AUTHTOKEN>
""
```

```
```bash
$ ngrok config add-authtoken <YOUR_AUTHTOKEN>
```
```

4. 新しいターミナルを開き、以下のコマンドを実行して `localhost:3000` をポートフォワーディングします。
- ※ コマンドを実行すると、`ngrok` が起動し、公開 URL が表示されます。この URL にアクセスすると、ローカルで動作している `localhost:3000` のコンテンツがインターネット経由で表示されます。

```
```PowerShell
PS> ngrok http 3000
```
```

```
```bash
$ ngrok http 3000
```
```



The screenshot shows the ngrok terminal output. At the top, it says "ngrok" in green. Below that, a message: "Using ngrok for OSS? Request a community license: https://ngrok.com/r/oss". Then, "Session Status" is shown in green, followed by "online" in green. Below that, "Account" is "username (Plan: Free)", "Version" is "3.25.1", and "Region" is "Japan (jp)". The "Web Interface" is "https://127.0.0.1:4040". The "Forwarding" section shows "https://xxx.ngrok-free.app" in a red box, with "公開URL" (Public URL) written in red above it. The "Connections" section shows a table with columns: ttl, opn, rt1, rt5, p50, p90, and values: 0, 0, 0.00, 0.00, 0.00, 0.00.

| ttl | opn | rt1 | rt5 | p50 | p90 |
|-----|-----|------|------|------|------|
| 0 | 0 | 0.00 | 0.00 | 0.00 | 0.00 |

図 17 : ターミナル (フォワーディング済み)